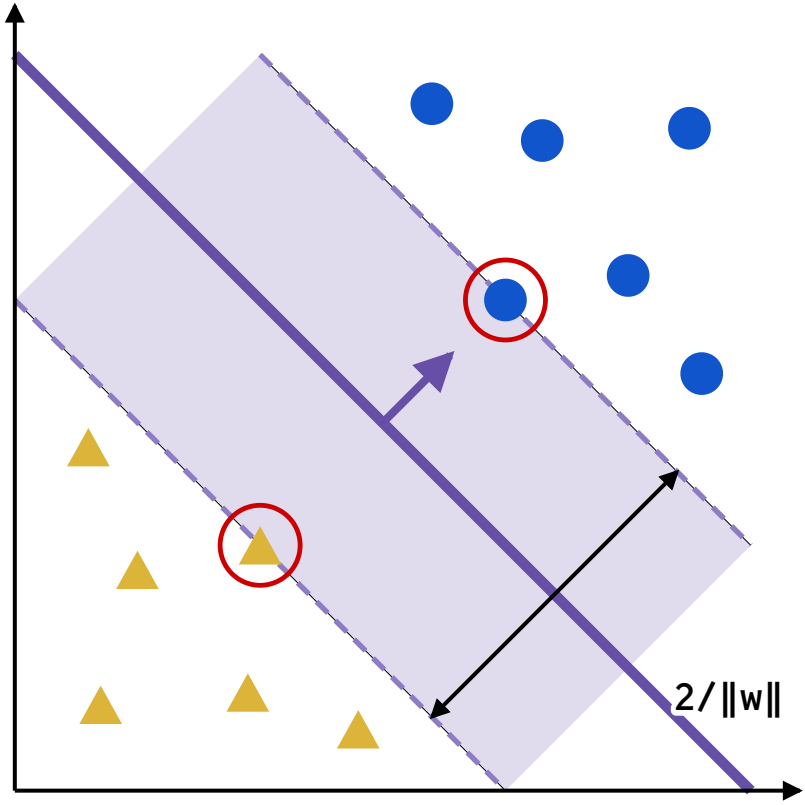


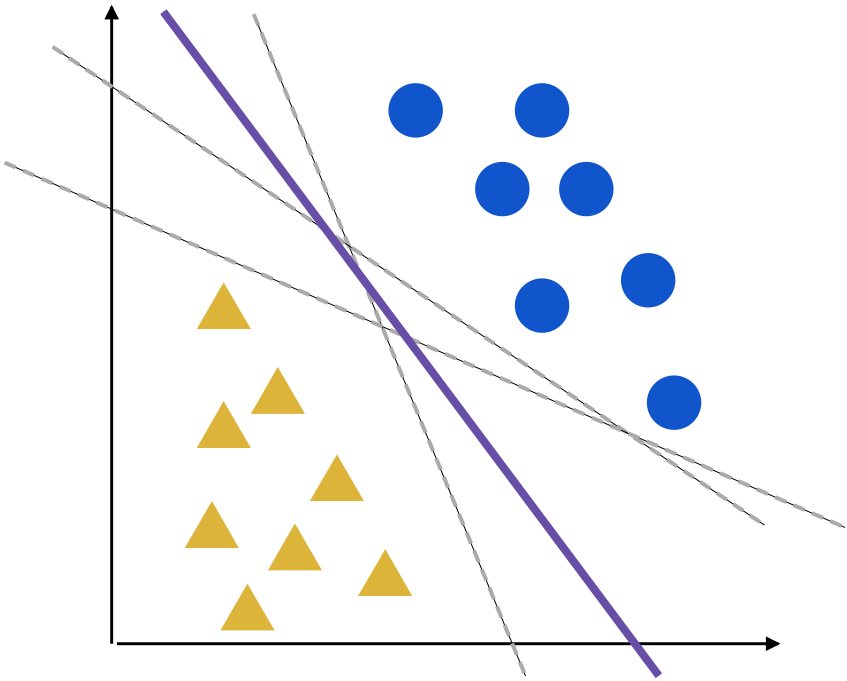

UNIT 02 · CHAPTER 6

# Support Vector Machines

CSE 30124 · Introduction to Artificial Intelligence
William Theisen · University of Notre Dame

# Which line?

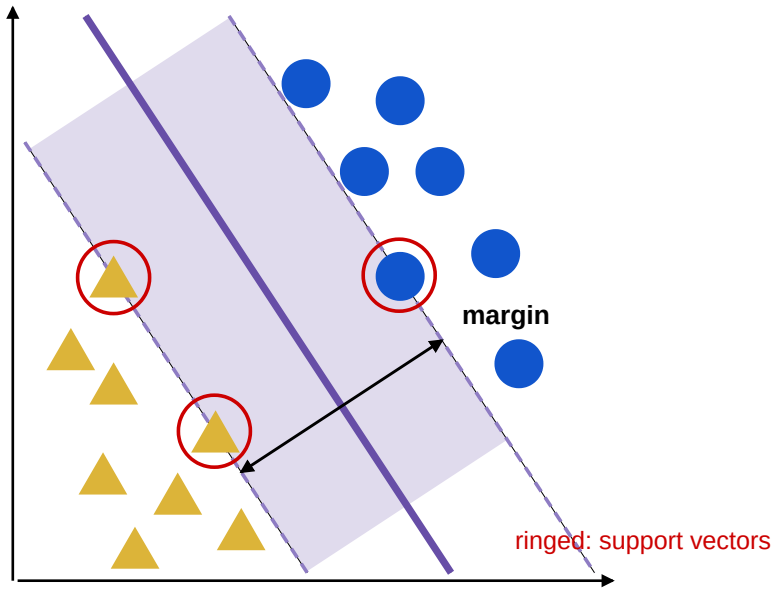
Let's revisit our **Perceptron** from the last chapter. If we train it on the data below, it will happily find us a line that separates the two classes. The problem is that **it finds us a line, not the line.** Different starting weights give different answers, and every one of them gets a perfect score on the training data!



So which one should we pick? Intuitively, we want the line that stays **as far from both classes as it can.** That way, if a new point shows up a little off from where we expected, we still classify it correctly.

# The widest street

The empty street between the two classes is called the **margin**. The best line runs down the middle of the widest street we can find. That's our next classifier, the **Support Vector Machine** (SVM)!




Notice that only the few points closest to the line matter. We call them **support vectors**, since they "hold up" the edges of the street. Move any other point around, as long as it stays off the street, and the line won't budge.

**Quick check:** slide a point that isn't ringed, keeping it off the street. Does the line move? What if you slide a ringed one?

# Where SVMs came from

The ideas in this chapter took about thirty years to come together, and most of them trace back to Vladimir Vapnik.

## 1963: The widest line

Working in Moscow, Vapnik and his colleagues proposed picking the separating line with the widest possible margin.

## 1992: Adding kernels

At AT&T Bell Labs, Boser, Guyon and Vapnik paired the widest margin with kernels, making curved boundaries cheap to learn.

## 1995: The soft margin

In "Support-Vector Networks," Corinna Cortes and Vapnik added slack, giving us the soft margin. For the next decade, SVMs were the go-to classifier for everything from handwritten digits to text.

# The road ahead

Six steps, each one fixing a problem the step before leaves open:


1. **The widest street** — pp. 2–11
   Pick the line with the most room on both sides.
2. **Soft margins** — pp. 12–15
   Let a few points cheat, for a price $C$.
3. **When no line works** — pp. 16–18
   Add features, and curves become straight lines.
4. **The dual** — pp. 19–26
   Rewrite the problem so data only appears in dot products.
5. **Kernels** — pp. 27–32
   Swap each dot product for a cheap similarity function.
6. **In practice** — pp. 33–38
   sklearn, tuning $C$ and $\gamma$, many classes, other models.


### Notation

| | |
|---|---|
| $x_i$ | one example (a row) |
| $y_i \in \{-1, +1\}$ | its label |
| $w$, $b$ | the weights (a row, as long as $x$) and the bias |
| $\hat{y}_i = x_i\, w^\top + b$ | the score, a number; its sign is the prediction |
| $2/\lVert w \rVert$ | the margin |
| $\varepsilon_i$, $C$ | slack, and the price of slack |
| $\alpha_i$, $K$ | point $i$'s multiplier (its knob), and a kernel |

# Scoring points

Just like our **Perceptron**, an SVM scores every point by multiplying its features by our weights and adding a bias. A positive score predicts class +1; a negative score predicts −1.

For a point $\mathbf{x}$, weights $\mathbf{w}$ (both rows of the same length) and a bias $\mathbf{b}$, the score is:

[6.1] score

$$\hat{y} = \mathbf{x}\mathbf{w}^{\top} + \mathbf{b}$$

The SVM adds two more lines, one on each side of the boundary. (With more features, lines become flat planes called **hyperplanes**.) Our goal: every point on or past the line for its class.

$$\begin{aligned} &\text{decision boundary:} && \mathbf{x}\mathbf{w}^{\top} + \mathbf{b} = 0 \\ &\text{positive hyperplane:} && \mathbf{x}\mathbf{w}^{\top} + \mathbf{b} = +1 \\ &\text{negative hyperplane:} && \mathbf{x}\mathbf{w}^{\top} + \mathbf{b} = -1 \end{aligned}$$

**Quick check:** $\mathbf{w} = [1, -1]$, $\mathbf{b} = 0.5$ and $\mathbf{x} = [2, 1]$. What is the score $\hat{y}$, and which class does the SVM predict?

# One rule for both classes

Checking "past the +1 line" for one class and "past the −1 line" for the other gets annoying. Here's a trick: multiply the score by the label $y$.

| Target $y$ | Score $\hat{y}$ | Product $y\hat{y}$ | Result |
|---|---|---|---|
| +1 | +2.3 | +2.3 | correct |
| +1 | +0.4 | +0.4 | close call |
| −1 | +0.7 | −0.7 | wrong |
| −1 | −1.8 | +1.8 | correct |

When the prediction is right, the product is positive, no matter which class the point is in! The bigger it is, the more room to spare. So one rule covers every point:

[6.2] every point on or past its line

$$y_i(x_i w^\top + b) \geq 1$$

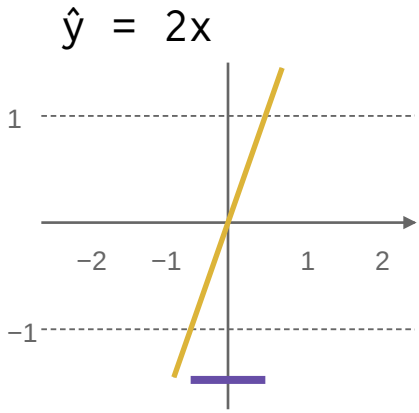
Requiring $y_i\hat{y}_i \geq 1$, not just $> 0$, means correct **and** not too close to the boundary. The +0.4 row is right, but it fails. Why 1? Next page.

# Why 1?

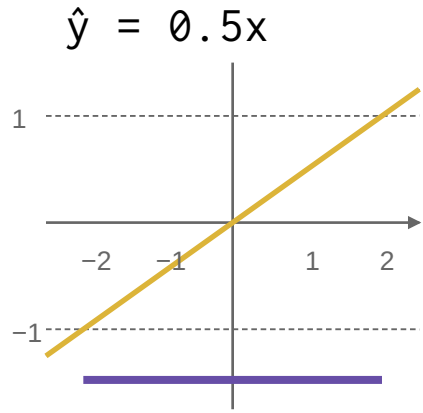
Why a score of 1, and not 5? If we double $\mathbf{w}$ and $\mathbf{b}$, every score doubles, but **the boundary doesn't move**:

$$\mathbf{x}\mathbf{w}^\top + \mathbf{b} = 0 \quad \Leftrightarrow \quad 2(\mathbf{x}\mathbf{w}^\top + \mathbf{b}) = 0$$

So the 1 itself is just a convention. What really matters is **how fast the score grows** as we walk away from the boundary:




width 1




width 4

With $\hat{y} = 2\mathbf{x}$, the ±1 lines are 1 apart; with $\hat{y} = 0.5\mathbf{x}$, they're 4 apart. **Smaller weights mean a wider margin.** The catch: every point still needs $y_i \hat{y}_i \geq 1$, and that's what stops $\mathbf{w}$ from shrinking to zero.

# How wide is the margin?

With more than one feature, the weight vector $\mathbf{w}$ points straight across the street. Sliding along the boundary doesn't change the score, so $\mathbf{w}$, the direction where the score grows fastest, has to be perpendicular to it.

If we take a step of length $t$ in that direction, the score changes by exactly $t\|\mathbf{w}\|$:

walk a distance t along w

$$\left(\mathbf{x} + t\,\frac{\mathbf{w}}{\|\mathbf{w}\|}\right)\mathbf{w}^T + b - (\mathbf{x}\mathbf{w}^T + b) = t\,\frac{\mathbf{w}\mathbf{w}^T}{\|\mathbf{w}\|}$$

$$\downarrow \text{ use } \mathbf{w}\mathbf{w}^T = \|\mathbf{w}\|^2$$

$$= t\,\frac{\|\mathbf{w}\|^2}{\|\mathbf{w}\|} = t\,\|\mathbf{w}\|$$

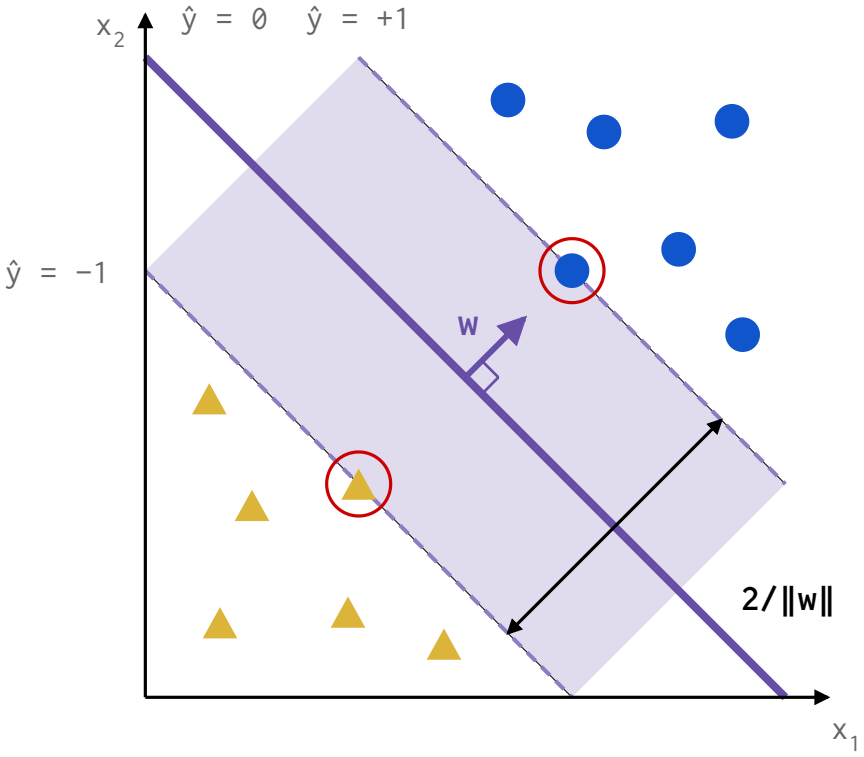
The ±1 lines are 2 apart in score, so $t\|\mathbf{w}\| = 2$. That distance, $2/\|\mathbf{w}\|$, is the **margin**: exactly the street from page 3. To widen it, we shrink $\mathbf{w}$:

[6.3] margin width

$$\max \frac{2}{\|\mathbf{w}\|} \quad \Leftrightarrow \quad \min \frac{1}{2}\|\mathbf{w}\|^2$$

# Picturing the margin

Here's a street like the one on page 3, drawn with two features. The boundary is where the score $\hat{y}$ is 0, and the dashed lines are where it's ±1:




$\mathbf{w}$ sticks straight out of the boundary. Walk along it from the −1 line to the +1 line and the score climbs by 2, which takes a distance of $2/\|\mathbf{w}\|$. **Shrink $\mathbf{w}$ and the street gets wider.**

# The hard-margin SVM

Putting it together gives the **hard-margin SVM**:

[6.4]

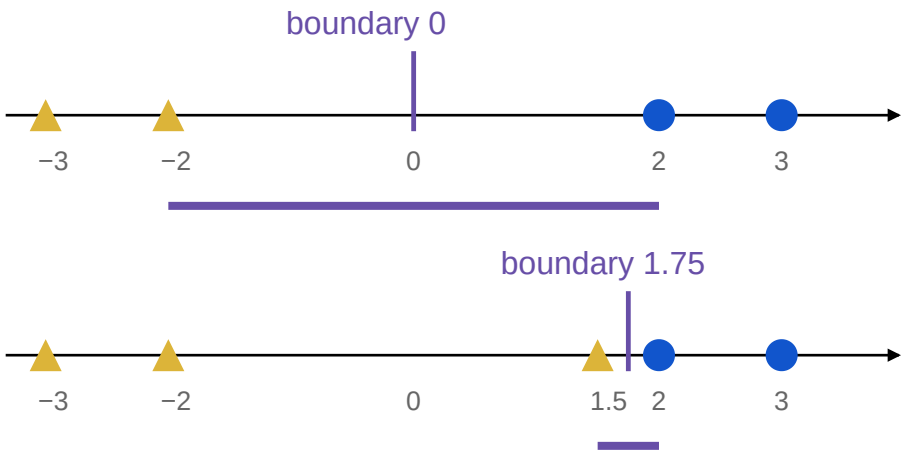
hard-margin SVM

$$\min_{w,b} \ \frac{1}{2}\|w\|^2$$

$$\downarrow \text{ subject to}$$

$$y_i(x_i w^T + b) \geq 1 \quad \text{for every } i$$

It has a big weakness:




On the top line, the closest points are at −2 and +2, so the boundary sits at 0. Add one more −1 point at 1.5 and **the margin shrinks from 4 to 0.5**!

Every point has to be on the correct side, so one bad point can wreck the whole classifier. And if the classes overlap at all, no line works and the hard-margin SVM has no answer.

# Soft margins

The fix is to let each point **cheat a little.** We give every point $i$ a **slack** value $\varepsilon_i$ that says how far it's allowed to fall short of its line:

[6.5] constraint with slack

$$y_i(x_i w^\top + b) \geq 1 - \varepsilon_i, \qquad \varepsilon_i \geq 0$$

But if cheating were free, every point would do it! So we charge for it, adding the total slack to what we minimize. $C$ sets the price of slack:

[6.6] soft-margin SVM

$$\min_{w,b,\varepsilon} \ \frac{1}{2}\|w\|^2 + C\sum_i \varepsilon_i$$

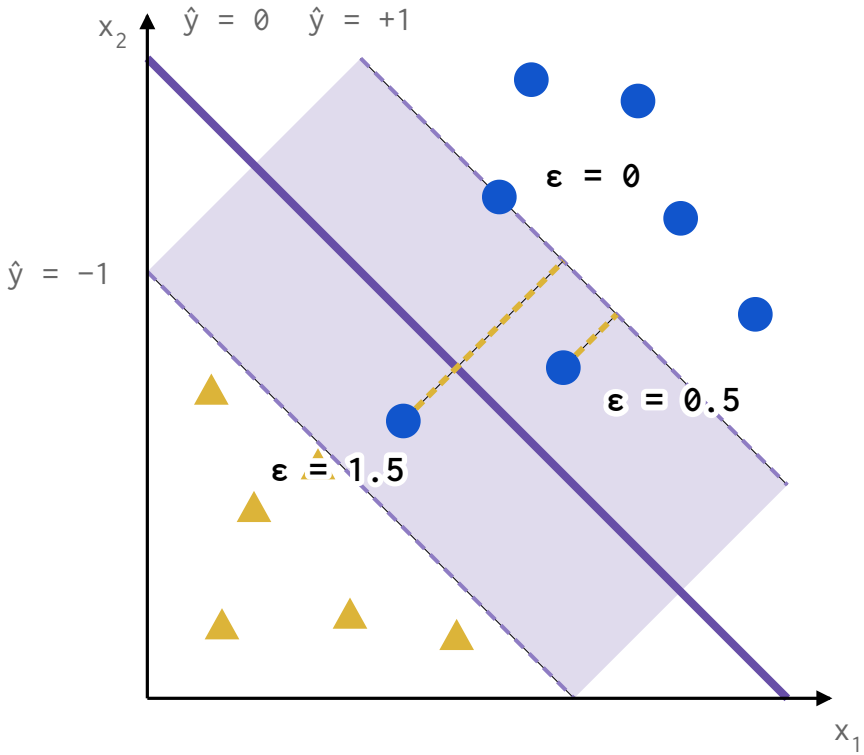
- $\varepsilon = 0$: on or past its line.
- $0 < \varepsilon < 1$: inside the street, still correct.
- $\varepsilon = 1$: right on the boundary.
- $\varepsilon > 1$: misclassified.

This is the **soft-margin SVM**, the version everyone actually uses.

**Quick check:** a point with target $+1$ scores $\hat{y} = 0.2$. How much slack does it need? What if it scores $-0.5$?

# Slack, pictured

Here's that street again, watching three circles. Each should sit on or past the +1 line. The dotted lines show how far short each one falls:




A circle on its line needs no slack. One inside the street needs a little ($0 < \varepsilon < 1$). One past the boundary needs **more than 1**, and that's exactly what being misclassified means.

# Turning the C knob

$C$ decides what we care about more: a wide margin or fewer violations. Say we're choosing between two lines:

| Candidate | $\frac{1}{2}\|w\|^2$ | Total slack |
|---|---|---|
| A: wider margin | 1 | 3 |
| B: fewer violations | 4 | 0.5 |

```
C = 0.1: A costs 1.3, B costs 4.05
C = 10:  A costs 31,  B costs 9
```

With a **small $C$**, slack is cheap, so we pick A and its wide margin. With a **large $C$**, slack is expensive, so we pick B and squeeze in more points.

Large $C$ acts like our hard-margin SVM and **can overfit**. Small $C$ tolerates more mistakes and **can underfit**. We pick $C$ using validation data.

# Hinge loss

There's another way to look at slack. For a given line, the smallest slack a point needs is:

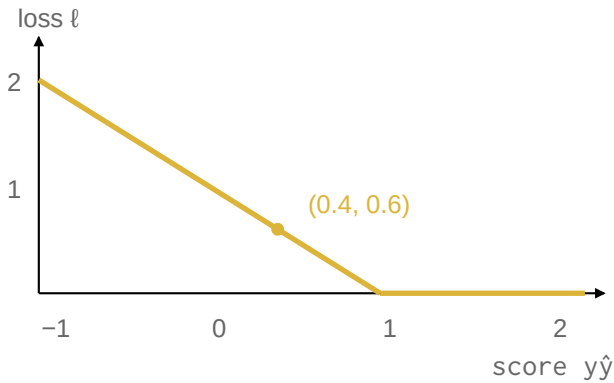
[6.7] hinge loss

$$\varepsilon_i = \max(0,\ 1 - y_i \hat{y}_i)$$

This is called **hinge loss**. Points past their line cost nothing; everyone else pays for how far short they fall. So an SVM is really minimizing:

[6.8] the same SVM, unconstrained

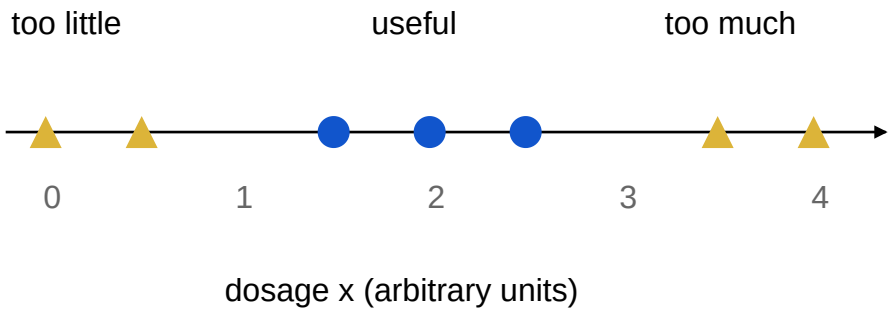
$$\min_{w,b} \ \frac{1}{2}\|w\|^2 + C \sum_i \max(0,\ 1 - y_i \hat{y}_i)$$




The marked point is the close call from page 7: it scores 0.4, so it pays 0.6. That's L2 regularization plus hinge loss: the same recipe as logistic regression! One twist: $C$ multiplies the loss, not the penalty, so **big $C$ means weak regularization**.

# When no line works

Soft margins handle a few messy points, but some data just isn't linear. Pretend we're testing a new drug: **too little does nothing, too much is harmful**, and only the middle dose is useful.

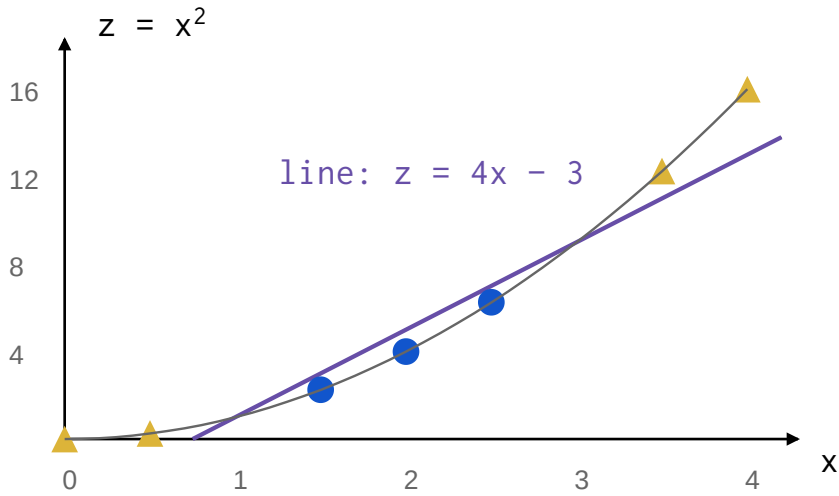



No matter where we put a single threshold, we get either the low doses or the high doses wrong. **One cut can't isolate the middle.**

Slack can't fix this. The problem is the shape of our data, not a few bad points.

# Adding a dimension

What if we add a second feature, $z = x^2$? Now our data sits on a parabola:




In this new 2D space, a straight line separates the classes! One line that works is $z = 4x - 3$, which gives the score:

$$\hat{y} = -3 + 4x - z = 1 - (x - 2)^2$$

Back in our original 1D space, that line becomes two thresholds, at $x = 1$ and $x = 3$. **A straight line up here is a curve down there.**

# Too many features

Adding $x^2$ worked, and we could even run gradient descent on hinge loss like logistic regression. So why not add features every time? Count the squares and products we'd need (degree 2), or cubes too (degree 3):

| Features we start with | Degree 2 | Degree 3 |
|---|---|---|
| 1 (our dosage) | 2 | 3 |
| 100 | 5,150 | 176,850 |
| 784 (an MNIST image) | 308,504 | 80.9 million |

Every one of those features has to be built and stored for every training point, and gradient descent needs a weight for each. The RBF kernel we'll meet later needs **infinitely many**!

The next few pages rewrite the SVM so the data only ever shows up in pairs, as **dot products**. Then we'll see that a dot product between two huge feature vectors can be cheap, even when the vectors themselves aren't.

# Lagrange multipliers

**Recap:** the hard-margin SVM [6.4] asks for $y_i(x_i\, w^\top + b) \geq 1$ at every point. Rules are awkward to optimize, so we fold each one into the cost. Point $i$ gets a **Lagrange multiplier** $\alpha_i \geq 0$, a knob that multiplies its rule. The **Lagrangian** $L$ depends on $w$, $b$ and every $\alpha_i$:

[6.9] Lagrangian

$$L = \frac{1}{2}\|w\|^2 - \sum_i \alpha_i \left[ y_i (x_i w^\top + b) - 1 \right]$$

Think of $L$ as a game: we pick $w$ and $b$ to make $L$ small, and the $\alpha$'s try to make it big. What each knob can do depends on where its point sits:

| Point's $y_i \hat{y}_i$ | Raising $\alpha_i$… | So $\alpha_i$… |
|---|---|---|
| > 1: room to spare | lowers $L$ | stays at 0 |
| = 1: on its line | leaves $L$ alone | can be > 0: a support vector |
| < 1: breaks its rule | sends $L$ to $\infty$ | so we must fix $w$, $b$ |

The only way for us to win is to satisfy every rule, and only the support vectors push back. Next: set $L$'s slopes to zero, plug the result back in, and we're left with a problem about the $\alpha$'s alone.

# Where the weights come from

$L$ is shaped like a saddle: we push it down in $w$ and $b$ while the $\alpha$'s push it up. For any fixed $\alpha$'s, the lowest point in $w$ and $b$ is where $L$ is flat, so we set those slopes to zero:

slope in w

$$\frac{\partial L}{\partial w} = w - \sum_i \alpha_i y_i x_i = 0$$

↓ solve for $w$

stationarity in w

$$[6.10] \quad w = \sum_i \alpha_i y_i x_i$$

slope in b

$$[6.11] \quad \frac{\partial L}{\partial b} = -\sum_i \alpha_i y_i = 0$$

[6.10] is huge: **the best weights are a weighted sum of our training points**, and only points with $\alpha_i > 0$ count. [6.11] says the pushes from each class balance out.

**Quick check:** $\alpha = [0.5, 0.5, 0]$ and $y = [+1, -1, +1]$. Does $\Sigma \alpha_i y_i = 0$ hold? Which points are support vectors?

# The dual problem

Now plug $w = \Sigma \alpha_i\, y_i\, x_i$ back into L, one piece at a time:

$$\tfrac{1}{2}\|w\|^2 = \tfrac{1}{2}\sum_i \sum_j \alpha_i \alpha_j y_i y_j x_i x_j^T$$

$$\sum_i \alpha_i y_i x_i w^T = \sum_i \sum_j \alpha_i \alpha_j y_i y_j x_i x_j^T$$

$$b \sum_i \alpha_i y_i = 0$$

$$L = \sum_i \alpha_i - \tfrac{1}{2}\sum_i \sum_j \alpha_i \alpha_j y_i y_j x_i x_j^T$$

The $w$'s and $b$ are gone! L depends on the $\alpha$'s alone, and they still want it big, so we maximize. This is the **dual problem**:

the dual problem

[6.12]

$$\max_{\alpha} \sum_i \alpha_i - \frac{1}{2}\sum_i \sum_j \alpha_i \alpha_j\, y_i y_j\, x_i x_j^T$$

$$\downarrow \text{ subject to}$$

$$\alpha_i \geq 0, \qquad \sum_i \alpha_i y_i = 0$$

Look closely: the data only shows up as dot products $x_i\, x_j^T$, **exactly the property we wanted!**

# Adding slack back

With slack, $L$ picks up two more pieces: the cost $C\Sigma\varepsilon_i$, and a new knob $\mu_i \geq 0$ for each rule $\varepsilon_i \geq 0$. Setting the slope in each $\varepsilon_i$ to zero:

two new pieces

$$L = \cdots + C\sum_i \varepsilon_i - \sum_i \alpha_i \varepsilon_i - \sum_i \mu_i \varepsilon_i$$

$$\downarrow \text{ set } \partial L/\partial \varepsilon_i = 0$$

$$C - \alpha_i - \mu_i = 0 \quad \Rightarrow \quad \alpha_i = C - \mu_i \leq C$$

Since $\mu_i \geq 0$, each $\alpha_i$ is capped at $C$. Everything else cancels exactly as before, so the soft-margin dual is the same problem with one extra limit:

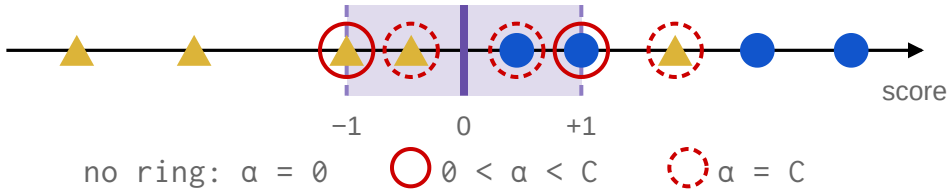
soft-margin dual constraints

$$[6.13] \quad 0 \leq \alpha_i \leq C, \qquad \sum_i \alpha_i y_i = 0$$

**Quick check:** at the answer, a point has $\alpha_i = C$. What is $\mu_i$? Is the point allowed to have slack?

# Finding support vectors

Once we solve for $\alpha$, the knob game sorts every point into one of three groups:




## $\alpha_i = 0$: not a support vector

The point clears its line with room to spare, so its knob stays off. It has no say in $w$, so we could delete it and get the same SVM!

## $0 < \alpha_i < C$: on the margin

The point sits exactly on its line, pushing just enough to hold it there. These are the classic support vectors.

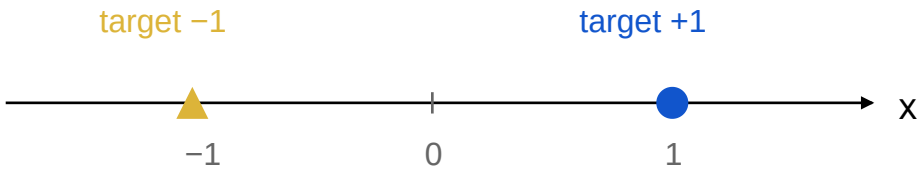
## $\alpha_i = C$: using slack

The point is inside the margin or misclassified. It pushes as hard as it's allowed: all the way to $C$.

# Solving one by hand

Let's make this concrete with the smallest dataset we can: two points, one feature, and $C = 1$.

$$\text{point 1:}\quad x_1 = -1,\quad y_1 = -1$$
$$\text{point 2:}\quad x_2 = +1,\quad y_2 = +1$$




The balance rule [6.11] says $-\alpha_1 + \alpha_2 = 0$, so both points get the same multiplier. Call it $a$. With one feature, the double sum in the dual is just $(\sum \alpha_i\, y_i\, x_i)^2$, and here each $y_i\, x_i = 1$:

the dual with both multipliers equal to a

$$q(a) = 2a - \frac{1}{2}\left(\sum_i a\, y_i x_i\right)^2$$

↓ each $y_i x_i = 1$

$$= 2a - \frac{1}{2}(a + a)^2 = 2a - 2a^2$$

Setting the derivative to zero, $2 - 4a = 0$, so $a = 0.5$. That's between 0 and $C$, so both points are support vectors sitting right on the margin.

# Recovering the line

Now we turn our $\alpha$'s back into weights using [6.10]:

$$w = \alpha_1 y_1 x_1 + \alpha_2 y_2 x_2$$
$$= 0.5(-1)(-1) + 0.5(+1)(+1) = 1$$

Then we find $b$ using point 1, which sits right on its line:

$$b = y_1 - x_1 w = -1 - (-1)(1) = 0$$

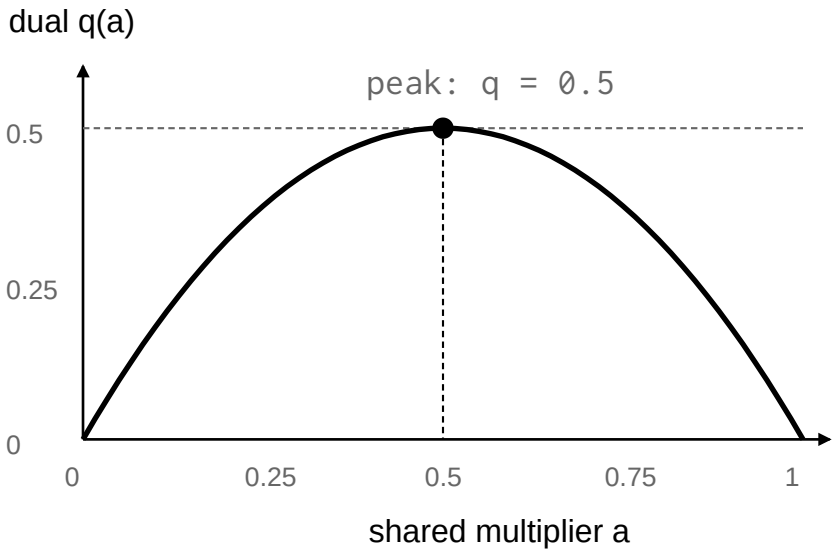



Our SVM is $\hat{y} = x$. The boundary is at 0, right between the points, and each point sits on its ±1 line. **The margin is $2/\|w\| = 2$**, as wide as it can be!

# Checking our answer

Here's the dual $q(a)$ for every $a$ between $0$ and $C$. Sure enough, it peaks at $a = 0.5$:

$$q(a) = 2a - 2a^2$$




The peak, 0.5, equals $\frac{1}{2}w^2$ for our line $w = 1$. Every dual value is a lower bound on the original (**primal**) cost, so when the two meet, we've found the best line.

$$\max \text{ dual} = 0.5 = \min \text{ original}$$

For SVMs they always meet, which is why we're allowed to solve the dual instead.

# The kernel trick

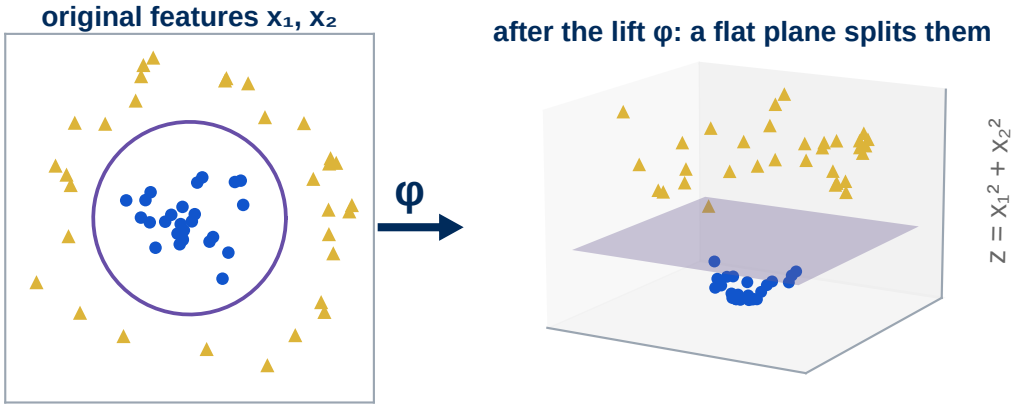
Remember how our dual only ever looked at the data through dot products, $x_i\ x_j^T$? That turns out to be a huge deal.

Call our feature transformation $\phi$ ("phi"). A **kernel** computes the new dot product directly:

[6.14] a kernel

$$K(x_i, x_j) = \phi(x_i)\,\phi(x_j)^T$$

Instead of building every new feature for every point, we just swap each dot product in the dual for K. That's the **kernel trick**!



# A kernel for our dosage data

Our dosage map keeps $x$ and adds $x^2$. Let's compare doses 2 and 3:

$$\phi(x) = [x, x^2]$$
$$\phi(2) = [2, 4], \quad \phi(3) = [3, 9]$$

Building the features first, their dot product is $2 \cdot 3 + 4 \cdot 9 = 42$. But we can get the same number straight from the doses:

$$K(x, x') = x\,x' + x^2 x'^2$$
$$K(2, 3) = 6 + 36 = 42$$

For one dose this saves nothing. The payoff comes with many features: a 28×28 MNIST image has 784 pixels. The degree-2 kernel $(x\,x'^{\mathsf{T}} + 1)^2$ needs one 784-long dot product. Building its features first means **over 300,000 numbers per image**!

# The kernel SVM

To get a kernel SVM, we take our dual and swap every dot product for K:

[6.15] the kernel SVM

$$\max_{\alpha} \sum_i \alpha_i - \frac{1}{2} \sum_i \sum_j \alpha_i \alpha_j y_i y_j K(x_i, x_j)$$

$$\downarrow \text{ subject to}$$

$$0 \le \alpha_i \le C, \qquad \sum_i \alpha_i y_i = 0$$

To classify a new point $x$, plug $w = \Sigma \alpha_i \, y_i \, \phi(x_i)$ into the score. We never build $w$ itself (for RBF, $\phi$ has infinitely many features!). Every dot product becomes a K:

[6.16] prediction

$$\hat{y}(x) = b + \sum_i \alpha_i y_i K(x, x_i)$$

Points with $\alpha_i = 0$ drop out of the sum, so **we only need to keep the support vectors** to make predictions. Compare that to **kNN**, which has to keep every single training point!

# Linear and polynomial kernels

## Linear: no transformation

[6.17] linear kernel

$$K(x, x') = x\,x'^{T}$$

This is just our original dot product, so we get the plain linear SVM from earlier in the chapter. It's a great first choice when you already have lots of features.

## Polynomial: add feature combinations

[6.18] polynomial kernel

$$K(x, x') = (\gamma\, x\, x'^{T} + r)^{p}$$

$p$ is the degree, $\gamma$ scales the dot product, and $r$ shifts it (sklearn calls $r$ `coef0`). For degree 2 with $u = [u_1, u_2]$, $v = [v_1, v_2]$, $\gamma = 1$ and $r = 0$:

$$(u_1 v_1 + u_2 v_2)^2 = u_1^2 v_1^2 + 2 u_1 u_2 v_1 v_2 + u_2^2 v_2^2$$

That's the dot product of $[u_1^2, \sqrt{2}u_1u_2, u_2^2]$ and $[v_1^2, \sqrt{2}v_1v_2, v_2^2]$. The kernel **includes the interaction term $u_1u_2$** without us ever building it.

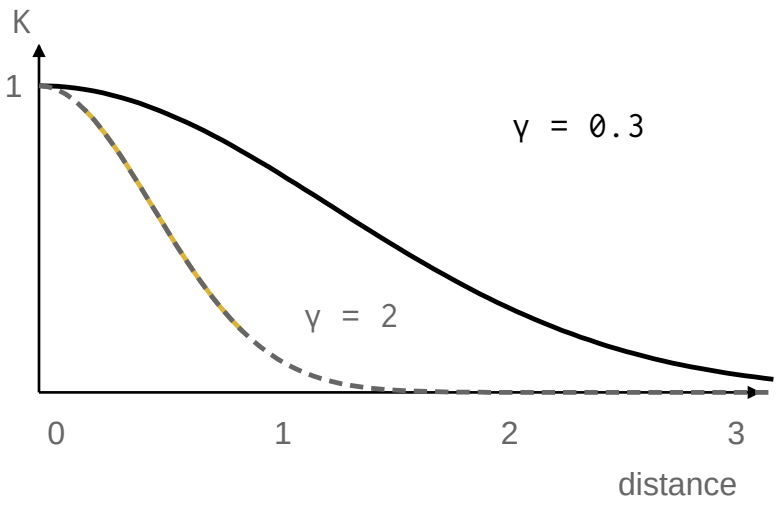
**Quick check:** with $\gamma = 1$ and $r = 0$, compute the degree-2 polynomial kernel for $u = [1, 2]$ and $v = [3, 1]$.

# The RBF kernel

The **radial basis function** (RBF) kernel measures how close two points are:

[6.19] RBF kernel

$$K(\mathbf{x}, \mathbf{x}') = \exp(-\gamma\|\mathbf{x} - \mathbf{x}'\|^2), \qquad \gamma > 0$$

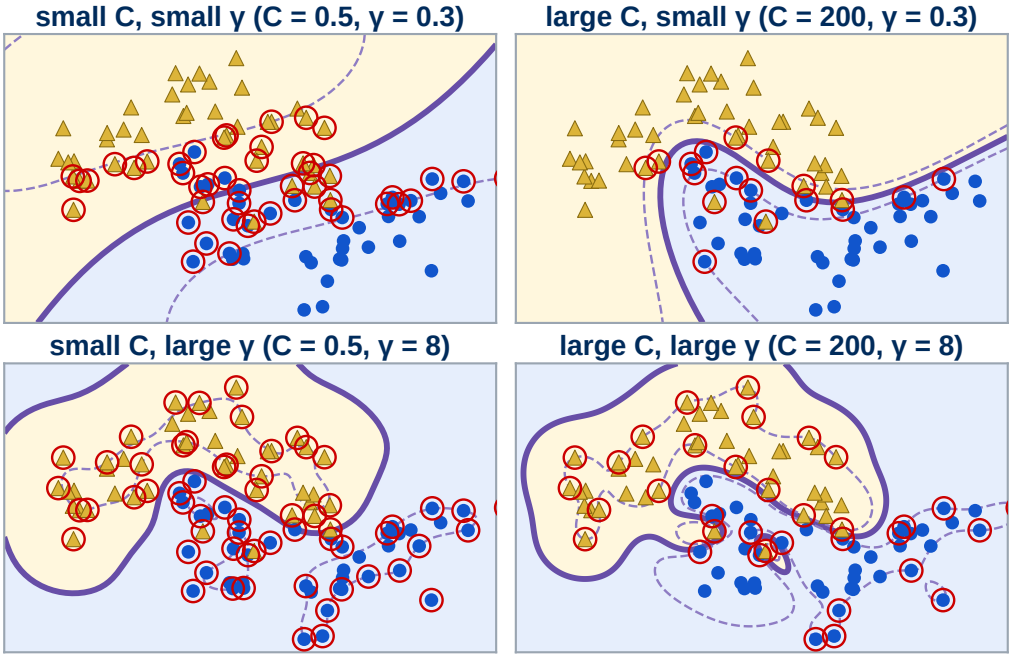



Identical points get $K = 1$, and $K$ drops toward 0 as points move apart. With $\gamma = 0.3$, points 1 apart score 0.74 and points 2 apart score 0.30.

$\gamma$ sets how far each point's influence reaches (a different job than in the polynomial kernel). Large $\gamma$ gives wiggly boundaries, small $\gamma$ smooth ones. **Scale your features first!**

# C and γ

An RBF SVM has two knobs, and they do different jobs. Here's the same data fit four ways (ringed points are support vectors):




Moving right, C grows: mistakes cost more, so the SVM bends to get more training points right. Moving down, γ grows: each support vector only shapes its own neighborhood, so the boundary wraps tightly around single points. The bottom right is **overfitting**.

Pick C and γ together with a grid search on validation data, never on training accuracy.

# SVMs in Python

We'll never solve the dual by hand for real data. Scikit-learn's `SVC` does it for us. Here's our dosage data from page 16:

```python
from sklearn.svm import SVC

# dosages, labeled useful (+1) or not (-1)
X = [[0], [0.5], [1.5], [2], [2.5], [3.5], [4]]
y = [-1, -1, 1, 1, 1, -1, -1]

# K(x, x') = (x*x' + 1)^2: gamma = 1, coef0 = r = 1
model = SVC(kernel="poly", degree=2, gamma=1,
            coef0=1, C=10)
model.fit(X, y)

print(model.support_vectors_.ravel())
print(model.predict([[2.2], [0.1]]))

# rebuild the score for x = 2.2 by hand
sv = model.support_vectors_.ravel()
a = model.dual_coef_.ravel()     # alpha_i * y_i
score = sum(a * (sv * 2.2 + 1)**2) + model.intercept_[0]
print(round(score, 3), model.decision_function([[2.2]]))
```

OUTPUT

```
[0.5 3.5 1.5 2.5]
[ 1 -1]
1.21 [1.20974964]
```

`support_vectors_` holds the points with $\alpha > 0$, and `dual_coef_` holds $\alpha_i\, y_i$. The last block rebuilds the score with `[6.16]`, and it matches sklearn. **Only 4 of our 7 points matter!**

# What did it learn?

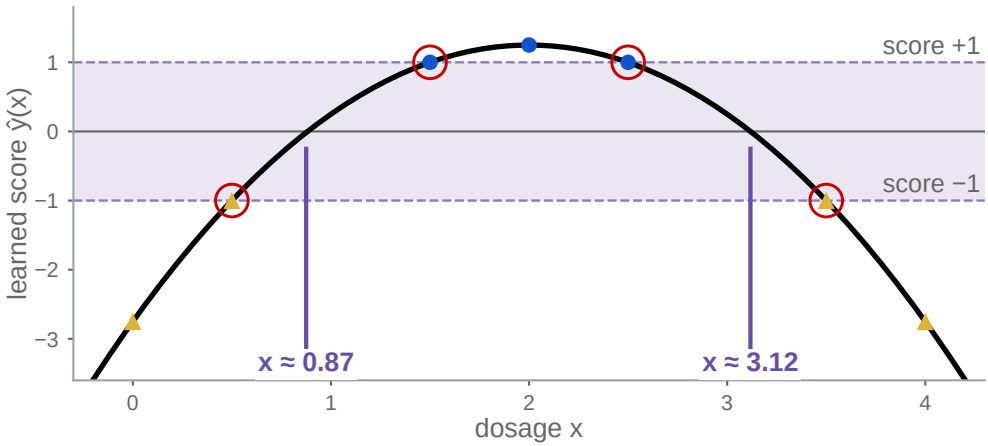
Our code used sklearn's degree-2 kernel $(x\,x' + 1)^2$, not the dosage kernel from page 28. Expanding it shows it builds our features $x$ and $x^2$ too, plus a constant:

sklearn's degree-2 kernel

$$(x\,x' + 1)^2 = 1 + 2\,x\,x' + x^2 x'^2$$

↓ read it as a dot product

$$= [1,\ \sqrt{2}\,x,\ x^2]\,[1,\ \sqrt{2}\,x',\ x'^2]^{\mathsf{T}}$$




The learned score is a parabola that crosses zero at $x \approx 0.87$ and $x \approx 3.12$, close to the thresholds 1 and 3 we picked by hand on page 17. The four ringed support vectors sit exactly on the ±1 lines. Slide any other point (off the street) and nothing changes.

# More than two classes

SVMs only answer yes/no questions. To handle $k$ classes, we train several SVMs and let them vote.

## One versus rest

Train $k$ SVMs, each separating one class from everything else. Predict the class whose SVM gives the highest score.

## One versus one

Train an SVM for every pair of classes. Each one votes, and the class with the most votes wins.

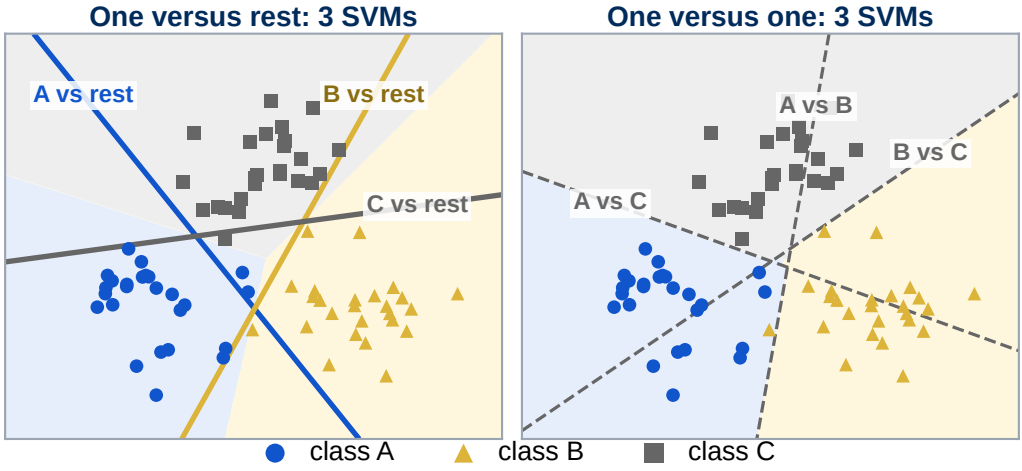
$$\text{one-vs-rest:} \quad k \text{ SVMs}$$

$$\text{one-vs-one:} \quad \frac{k(k-1)}{2} \text{ SVMs}$$

For MNIST digits ($k = 10$), that's 10 or 45 SVMs. One versus one trains more models, but each is small and fast since it only sees two classes.

# Seeing the votes

Here are both strategies on the same three classes:

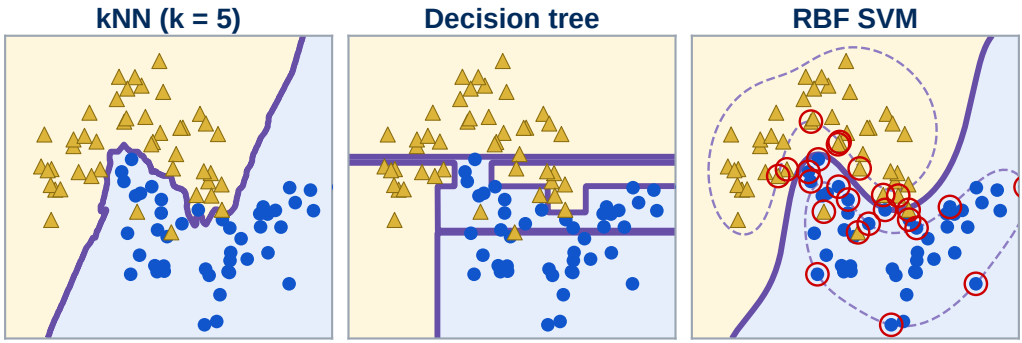



One versus rest trains one SVM per class, and each line tries to split its class off from everyone else (no single line manages it cleanly for A). One versus one trains one SVM per pair. Neither set of lines is the final answer: the shaded regions come from comparing scores (OvR) or counting votes (OvO).

**Quick check:** with 3 classes both strategies happen to train 3 SVMs. How many does each train for 5 classes?

# SVMs vs. kNN vs. trees

We've now seen three very different classifiers. Here's how each one splits the same data:




**k-nearest neighbors** keeps every training point and votes at prediction time. Its boundary can be as wiggly as the data, but predictions slow down as the dataset grows.

**Decision trees** split on one feature at a time, so their boundaries are built from boxes. Easy to read, but easy to overfit.

**SVMs** keep only their support vectors (ringed). With an RBF kernel the boundary looks a lot like kNN's, but smoothed out by the margin.

**No one model wins everywhere.** Try them on validation data and pick the winner.

# Cheat sheet

## The whole SVM

primal

$$[6.8] \quad \min_{w,b} \ \tfrac{1}{2}\|w\|^2 + C\sum_i \max(0,\ 1 - y_i\hat{y}_i)$$

dual

$$[6.15] \quad \max_{\alpha} \ \sum_i \alpha_i - \tfrac{1}{2}\sum_i\sum_j \alpha_i\alpha_j y_i y_j K(x_i, x_j)$$

$$\downarrow \text{ subject to}$$

$$0 \le \alpha_i \le C, \quad \sum_i \alpha_i y_i = 0$$

prediction

$$[6.16] \quad \hat{y}(x) = b + \sum_i \alpha_i y_i K(x, x_i)$$

## What $\alpha$ tells you

- $\alpha_i = 0$**:** past its line. It has no say in the model.
- $0 < \alpha_i < C$**:** exactly on its line. Use it to find $b$.
- $\alpha_i = C$**:** inside the street, or misclassified.

## Kernels

- **Linear** $x\,x'^{T}$: you already have lots of features.
- **Polynomial** $(\gamma\, x\, x'^{T} + r)^p$: feature interactions.
- **RBF** $\exp(-\gamma\|x - x'\|^2)$: the flexible default.

## Knobs

- **Bigger** $C$**:** slack costs more, narrower street, can overfit.
- **Bigger** $\gamma$**:** more local, wigglier boundary, can overfit.
- Scale features first. Tune $C$ and $\gamma$ together on validation data.

# Practice problems

1. **Correct but penalized.** A point has $y = +1$ and score 0.25. Is it classified correctly? How much slack does it need, and what does that cost with $C = 4$?
2. **Margin width.** A linear SVM has weights $w = [6, 8]$. How wide is its margin? If we double $w$ and $b$, does the boundary move? What happens to the margin?
3. **Support vectors.** A point sits far past its line. What is its $\alpha$? What about a point that's misclassified?
4. **Solving by hand.** Redo page 24 with $x_1 = -2$, $y_1 = -1$, $x_2 = +2$, $y_2 = +1$ and $C = 1$. Find $\alpha$, $w$, $b$ and the margin width.
5. **Kernels by hand.** Let $\phi([u_1, u_2]) = [u_1^2, u_2^2]$. Compute $K([1, 3], [2, 1])$. Is it the degree-2 polynomial kernel? Why not?
6. **Two knobs.** Your RBF SVM gets 100% training accuracy but only 60% on validation. Should you raise or lower $C$ and $\gamma$?
7. **More classes.** How many SVMs do one-versus-rest and one-versus-one train for 4 classes?

# Answers

1. $$y\hat{y} = 0.25, \qquad \varepsilon = 0.75, \qquad C\varepsilon = 3$$

   Positive, so it's correct, but it's inside the street: it needs 0.75 slack, which costs 3.

2. $\|w\| = 10$, so the margin is $2/10 = 0.2$. Doubling doesn't move the boundary, but $\|w\|$ becomes 20 and the margin shrinks to 0.1.

3. Far past its line: $\alpha = 0$, not a support vector. Misclassified: it needs slack, so $\alpha = C$.

4. Balance gives $\alpha_1 = \alpha_2 = a$ and $w = 4a$, so $q(a) = 2a - 8a^2$, peaking at $a = 1/8$. Then $w = 0.5$, $b = 0$, and the width is $2/0.5 = 4$: exactly the gap.

5. $$\begin{aligned} &\text{feature map:} && 1^2 \cdot 2^2 + 3^2 \cdot 1^2 = 13 \\ &\text{polynomial:} && ([1, 3][2, 1]^\top)^2 = 25 \end{aligned}$$

   They differ: the polynomial kernel also has the cross term $2u_1u_2v_1v_2$.

6. We're overfitting, so lower $C$ and $\gamma$ and check validation accuracy again.

7. 4 one-versus-rest SVMs, or $4 \cdot 3/2 = 6$ one-versus-one.

## Quick checks

# Companion materials

Both visualizers run in the browser; nothing to install.

## SVM visualizer

Explore linear and nonlinear datasets, decision boundaries, margins and support vectors.

**SVM visualizer**
Interactive, in the web edition: ai.williamtheisen.com/svm.html

## Regularization visualizer

Compare the geometric effects of L1 and L2 penalties and the role of regularization strength.

**Regularization visualizer**
Interactive, in the web edition: ai.williamtheisen.com/regularization.html